

Tema 4.

Análisis automático de textos para revisión lingüística

Informe automático de divergencias en textos paralelos monolingües

Material teórico | Programación para PLN | Universidad Pontificia Comillas

Dr.^a Blanca Hernández Pardo

Índice de contenidos

Título narrativo.....	1
Contexto profesional.....	1
Qué vas a aprender en esta unidad.....	2
Mapa conceptual de la unidad.....	2
1. Marco teórico.....	3
1.1. Textos paralelos monolingües.....	3
1.2. Por qué automatizar parte de la revisión.....	3
1.3. Del corpus al informe: una cadena de análisis.....	4
1.4. Revisión lingüística, calidad y evidencia.....	5
1.5. Corpus, texto y unidad de análisis.....	6
1.6. Segmentación en oraciones.....	6
1.7. Frecuencia léxica y coherencia terminológica.....	7
1.8. Patrones formales: espacios, puntuación y normalización.....	7
1.9. Expresiones regulares como reglas transparentes.....	8
1.10. Posibles calcos y estructuras con a + infinitivo.....	9
1.11. Gerundios como candidatos a revisión.....	9
1.12. Falsos positivos, falsos negativos y criterio lingüístico.....	10
1.13. Funciones, informes y trazabilidad.....	10
1.14. Límites de los métodos basados en reglas.....	11
2. Análisis automático de textos para revisión lingüística con Python.....	11
2.1. Leer textos como datos.....	11
2.2. Rutas de archivos y pathlib.....	12
2.3. Cadenas, listas y corpus pequeño.....	13
2.4. Segmentar en oraciones con split y strip.....	13
2.5. Contar palabras y normalizar con minúsculas.....	14
2.6. Recorrer varias versiones con enumerate.....	15
2.7. Detectar ausencia de espacio después de puntuación.....	16
2.8. Recuperar contexto con índices y slicing.....	17
2.9. Localizar dobles espacios.....	17
2.10. Comparación terminológica básica.....	18

2.11. Tablas de resultados.....	20
2.11. Comparar términos en muchas versiones.....	20
2.12. Posibles galicismos: Buscar candidatos a + infinitivo.....	21
2.13. Buscar gerundios y contexto inmediato.....	22
2.14. Organizar el análisis con funciones.....	24
2.15. Devolver resultados con `return`.....	26
2.16. Generar un informe final.....	27
2.17. Cómo leer los resultados antes de corregir.....	28
Reglas simples frente a análisis lingüístico avanzado.....	28
Buenas prácticas para revisar textos con Python.....	29
Errores frecuentes de aprendizaje.....	29
Síntesis final.....	30
Cierre narrativo.....	30
Referencias.....	30

Título narrativo

Revisar dos versiones de un mismo texto ya no consiste solo en leer con atención: consiste en saber convertir el texto en datos, detectar indicios y devolver al equipo humano una lista razonada de lugares que merecen revisión. En esta unidad, la revisión lingüística se plantea como una tarea profesional híbrida: el criterio sigue siendo humano, pero Python ayuda a localizar de forma rápida patrones repetitivos, incoherencias terminológicas y señales formales que pueden pasar inadvertidas cuando el volumen de texto crece.

Contexto profesional

Imagina que te incorporas al equipo lingüístico de una institución multilateral, un organismo público o una gran empresa que trabaja con documentación institucional en varias lenguas: **Verba Pública**. La institución ha encargado la traducción de un documento oficial a varios proveedores. Las versiones llegan en el mismo idioma, pero no son idénticas: una habla de **equipo** donde otra habla de **grupo**; una prefiere **compañía** y otra **empresa**; una mantiene una puntuación impecable y otra acumula pequeños problemas de espaciado.

La pregunta profesional no es únicamente cuál de las versiones «suena mejor». El equipo necesita tomar decisiones justificables:

- Qué versión puede validarse como base.
- Qué términos deben incorporarse a un glosario consolidado.
- Qué proveedor respeta mejor la política terminológica de la institución.
- Qué problemas formales conviene corregir antes de publicar el documento.
- Qué patrones lingüísticos necesitan revisión humana especializada.

La institución no quiere una lectura subjetiva aislada, sino un procedimiento repetible que ayude a responder preguntas concretas:

- ¿Hay divergencias terminológicas entre versiones?
- ¿Aparecen patrones formales que sugieran falta de revisión, como espacios dobles o ausencia de espacio tras puntuación?
- ¿Se repiten estructuras que podrían requerir criterio lingüístico, como ciertos usos de *a* + infinitivo o gerundios?
- ¿Puede generarse un pequeño informe que reúna los hallazgos sin tener que revisar manualmente todo el corpus desde cero?

En una revisión manual tradicional, una lingüista tendría que leer las versiones una al lado de la otra, oración por oración, y marcar diferencias. Esa lectura sigue siendo imprescindible, porque interpretar un texto exige criterio lingüístico. Sin embargo, cuando el volumen crece, la revisión exclusivamente manual se vuelve lenta, costosa y difícil de documentar.

Python permite construir una primera capa de revisión automática. Esa capa no reemplaza la lectura profesional, pero ayuda a organizarla. Localiza indicios, cuenta fenómenos, compara usos y genera un informe preliminar que orienta el trabajo humano. La idea central de este tema es muy importante para el PLN: un texto puede leerse como discurso, pero también puede representarse como dato.

Este manual desarrolla esa mirada: cómo preparar un pequeño sistema de análisis textual para revisar versiones paralelas monolingües, detectar divergencias terminológicas y localizar posibles problemas de estilo, puntuación y gramática.

Qué vas a aprender en esta unidad

Al terminar esta unidad deberías poder explicar, antes de ejecutar código, por qué cada operación es relevante para una tarea de revisión lingüística. En concreto, aprenderás a:

- entender qué son los textos paralelos monolingües en el contexto de una revisión comparativa;
- distinguir entre indicio automático, error lingüístico y decisión profesional;
- leer archivos de texto en Python controlando la codificación;
- representar un corpus pequeño como una lista de textos;
- segmentar textos en oraciones con procedimientos simples y reconocer sus limitaciones;
- normalizar texto con operaciones como minúsculas para facilitar comparaciones;
- contar palabras o expresiones como primera aproximación a la coherencia terminológica;
- recorrer muchos textos con bucles y `enumerate`;
- usar expresiones regulares para localizar patrones formales;
- recuperar contexto alrededor de una coincidencia;
- comparar versiones paralelas oración a oración;
- organizar comprobaciones en funciones reutilizables;
- diferenciar entre mostrar un resultado con `print` y devolverlo con `return`;
- reunir varios análisis parciales en un informe final interpretable.

Mapa conceptual de la unidad

Bloque	Pregunta lingüística	Herramienta de Python	Resultado esperado
Textos paralelos monolingües	¿Cómo comparo dos versiones españolas de un mismo contenido?	Listas, índices, zip	Correspondencia entre segmentos
Lectura de archivos	¿Cómo convierto documentos externos en texto procesable?	<code>open</code> , <code>encoding="utf-8"</code> , <code>pathlib.Path</code>	Cadenas de texto cargadas en memoria
Segmentación	¿Dónde empieza y termina cada unidad revisable?	<code>split</code> , <code>strip</code> , listas por comprensión	Lista de oraciones o segmentos
Frecuencia léxica	¿Qué términos se repiten y en qué versiones?	<code>lower</code> , <code>count</code> , bucles	Recuentos comparables
Patrones formales	¿Dónde hay señales mecánicas de posible error?	<code>in</code> , <code>finditer</code> , índices, slicing	Posiciones y contexto
Expresiones regulares	¿Cómo busco estructuras textuales flexibles?	Módulo <code>re</code> , patrones, grupos	Coincidencias interpretables
Funciones	¿Cómo evito repetir análisis manualmente?	<code>def</code> , parámetros, <code>return</code>	Comprobaciones reutilizables
Informe	¿Cómo convierto hallazgos dispersos en evidencia?	Funciones combinadas, tablas, texto formateado	Informe de revisión preliminar

1. Marco teórico

1.1. Textos paralelos monolingües

En corpus lingüísticos y estudios de traducción, un corpus paralelo suele entenderse como un conjunto de textos alineados en dos o más lenguas, normalmente un original y su traducción. Un corpus comparable, en cambio, reúne textos que comparten rasgos relevantes, como dominio, género o periodo, pero no necesariamente son traducciones directas entre sí (Olohan, 2004; Zanettin, 2012). La unidad trabaja con una adaptación pedagógica de esa lógica: **textos paralelos monolingües**, es decir, versiones en la misma lengua que representan un mismo contenido o una misma función comunicativa y que pueden compararse por secciones, párrafos u oraciones.

La expresión «paralelo monolingüe» debe entenderse aquí con cuidado. No significa que exista una categoría universal cerrada con ese nombre en todas las tradiciones de corpus, sino que la unidad aprovecha la idea de paralelismo textual para una tarea profesional concreta: comparar versiones españolas de un mismo documento. En revisión institucional esto es habitual. Puede haber dos proveedores que traducen o adaptan un texto, dos versiones sucesivas de un informe, una versión revisada y otra sin revisar, o variantes regionales de un mismo contenido.

El interés lingüístico de estos textos está en la divergencia. Si dos versiones dicen exactamente lo mismo con las mismas palabras, la comparación automática aporta poco. Si difieren, en cambio, esas diferencias pueden revelar decisiones de estilo, terminología, registro, puntuación o estructura sintáctica. La tarea no consiste en asumir que toda diferencia es mala. La variación es normal y, en muchos casos, legítima. Lo importante es distinguir entre variación aceptable, incoherencia terminológica y posible error.

Tipo de divergencia	Ejemplo	Interpretación posible
Terminológica	«equipo» frente a «grupo»	Puede ser variación aceptable o falta de glosario común
Ortográfica o mecánica	«empresa,el»	Probable problema formal por ausencia de espacio
Estilística	«llevar a cabo» frente a «realizar»	Puede responder a preferencias de estilo
Sintáctica	«decidió a revisar»	Puede ser candidato a revisión según contexto
Discursiva	Una versión divide en dos oraciones y otra en una	Puede afectar claridad o ritmo

Para una lingüista computacional en formación, el punto clave es que el texto se convierte en un objeto analizable. Una versión puede representarse como una cadena de caracteres; varias versiones pueden guardarse en una lista; cada oración puede ocupar una posición; cada término puede contarse; cada patrón puede buscarse. Esa representación no agota el análisis lingüístico, pero lo hace operativo.

1.2. Por qué automatizar parte de la revisión

La revisión lingüística y la revisión de traducción son tareas expertas que requieren comprensión, competencia textual, conocimiento del dominio y criterio profesional. Las normas de servicios de traducción insisten en procesos de revisión y verificación que no se reducen a una comprobación mecánica (ISO, 2015). La evaluación de la calidad de una traducción también exige criterios definidos y categorías de error o adecuación, no solo impresiones generales (ISO, 2024; Lommel et al., 2014).

Automatizar parte de la revisión no significa automatizar la decisión final. Significa delegar en el programa las operaciones que son repetitivas, sistemáticas y fácilmente verificables. Python puede contar cuántas

veces aparece un término, localizar dobles espacios, detectar una coma sin espacio posterior o listar estructuras candidatas a revisión. La persona especialista conserva la responsabilidad de interpretar esos hallazgos.

Este reparto de tareas tiene varias ventajas. En primer lugar, reduce la carga de búsqueda manual. En un texto largo, encontrar todas las ocurrencias de una palabra o todos los espacios dobles es una tarea tediosa y propensa a omisiones. En segundo lugar, aumenta la trazabilidad: si se usa el mismo patrón de búsqueda, el procedimiento puede repetirse en otra versión o en otro corpus. En tercer lugar, facilita la documentación de decisiones: un informe con recuentos, ejemplos y posiciones permite discutir con más precisión.

También existen riesgos. Un detector basado en reglas puede producir falsos positivos, es decir, casos señalados por el programa que no son errores reales. También puede producir falsos negativos, casos problemáticos que no se detectan porque no cumplen el patrón previsto. Por eso conviene hablar de **candidatos a revisión** y no de errores automáticos. El lenguaje es demasiado variable para que una regla simple capture todos los matices.

En Verba Pública, esta idea se traduce en una regla profesional clara: el informe automático no se entrega como veredicto, sino como mapa de lectura. La revisora sénior no recibe una lista de «fallos», sino una lista de «lugares que conviene mirar». Esa diferencia protege la calidad del trabajo lingüístico y evita que la herramienta aparente más certeza de la que realmente tiene.

1.3. Del corpus al informe: una cadena de análisis

Una tarea de PLN no empieza con el informe final, sino con una cadena de decisiones. En esta unidad, esa cadena puede resumirse así: recopilar textos, leerlos correctamente, normalizarlos cuando sea necesario, segmentarlos, aplicar comprobaciones, guardar resultados, interpretarlos y comunicarlos. Cada eslabón condiciona los siguientes.

Fase	Pregunta	Decisión técnica	Riesgo si se descuida
Entrada	¿De dónde viene el texto?	Leer archivos .txt con codificación explícita	Caracteres dañados o errores de lectura
Representación	¿Cómo guardo una o varias versiones?	Cadenas, listas, diccionarios si procede	Datos desordenados o difíciles de comparar
Normalización	¿Qué diferencias quiero ignorar?	Pasar a minúsculas, limpiar espacios	Comparaciones injustas o pérdida de información
Segmentación	¿Qué unidad reviso?	Oraciones, párrafos, líneas	Coincidencias sin contexto
Análisis	¿Qué busco?	Recuentos, patrones, regex, funciones	Resultados irrelevantes
Interpretación	¿Qué significa el hallazgo?	Criterio lingüístico	Confundir señal con error
Informe	¿Cómo lo comunico?	Tablas, ejemplos, resumen	Evidencia difícil de usar

La literatura de corpus insiste en que el corpus no es una colección neutra de textos, sino un recurso construido para responder preguntas concretas (McEnery y Hardie, 2012). En una revisión profesional, esa idea es especialmente importante: no se recopilan versiones porque sí, sino porque se quiere verificar coherencia, detectar divergencias y justificar decisiones.

El informe final debe preservar la relación entre dato y contexto. Un recuento aislado puede ser útil, pero no basta. Saber que «implementación» aparece cinco veces en una versión y una vez en otra puede sugerir divergencia terminológica, pero la revisión necesita mirar dónde aparece, qué función cumple y si existe una alternativa aceptada por el glosario. Por eso el análisis automático debe tender a producir resultados contextualizados.

Por ejemplo, si una herramienta detecta diez casos de a + infinitivo, no puede afirmar automáticamente que todos sean galicismos. Algunos serán correctos. Otros dependerán del sustantivo anterior, del régimen preposicional o de la construcción completa. La herramienta reduce el espacio de búsqueda; la lingüista interpreta.

Esta relación entre automatización y revisión humana es esencial en entornos profesionales:

Tarea automática	Decisión lingüística posterior
Contar ocurrencias de empresa y compañía	Decidir cuál debe incorporarse al glosario
Detectar ., seguido de una letra sin espacio	Confirmar si hay error tipográfico real
Localizar gerundios	Clasificar usos correctos, de posterioridad o especificativos
Comparar términos por versión	Valorar consistencia, estilo y adecuación al cliente
Generar un informe preliminar	Priorizar la revisión humana

Para una lingüista de segundo curso, esta unidad introduce una idea muy poderosa: programar no consiste solo en «hacer que algo funcione». Programar para PLN consiste en **formalizar una pregunta lingüística**.

1.4. Revisión lingüística, calidad y evidencia

La calidad textual no es una propiedad única. En traducción y comunicación especializada suele relacionarse con adecuación al encargo, fidelidad funcional, coherencia terminológica, corrección lingüística, legibilidad y cumplimiento de convenciones del cliente o institución. Las normas profesionales, como ISO 17100 para servicios de traducción, subrayan la importancia de procesos definidos, competencias profesionales y revisión por una persona distinta cuando procede (ISO, 2015).

La evaluación de calidad también puede apoyarse en marcos de categorías. MQM, por ejemplo, propone describir problemas de calidad mediante dimensiones y tipos de error, lo que permite pasar de una impresión global a una anotación más sistemática (Lommel et al., 2014). ISO 5060 actualiza este terreno al ofrecer una norma para evaluar resultados de traducción, con atención a criterios, especificaciones y finalidad de la evaluación (ISO, 2024).

La unidad no pretende que el alumnado aplique una taxonomía completa de calidad, sino que entienda una idea fundamental: un buen análisis automático debe estar al servicio de una pregunta de revisión. No se busca «todo lo raro»; se buscan indicios relacionados con una necesidad profesional. En el caso de Verba Pública, la necesidad es comparar versiones españolas para detectar divergencias antes de consolidar una versión final.

Esta perspectiva evita dos errores frecuentes. El primero es tecnicista: creer que si Python detecta algo, ese algo ya es un error. El segundo es antitécnico: pensar que, como el criterio final es humano, la automatización no aporta valor. Entre ambos extremos está la práctica profesional madura: usar herramientas para observar mejor y decidir con más fundamento.

1.5. Corpus, texto y unidad de análisis

Un corpus es un conjunto de textos reunidos con un propósito. Puede ser grande o pequeño, especializado o general, monolingüe o multilingüe, equilibrado o diseñado para una tarea muy concreta. Lo importante es que permite estudiar patrones de uso a partir de datos. En PLN, un corpus pequeño puede ser suficiente para aprender procedimientos; en investigación o producción profesional, el tamaño y diseño del corpus deben justificarse con más cuidado (McEnery y Hardie, 2012; Olohan, 2004).

En esta unidad, el corpus es pequeño y controlado: varias versiones de textos de trabajo. Esa escala es didáctica, pero no trivial. Muchas tareas profesionales reales comienzan con corpus modestos: un contrato, un informe, una guía de estilo, una batería de páginas web o un conjunto de documentos institucionales. En esos casos, Python no se usa para descubrir leyes generales de la lengua, sino para apoyar una revisión concreta.

La unidad de análisis puede cambiar. A veces interesa el texto completo, por ejemplo para contar cuántas veces aparece un término. Otras veces interesa la oración, porque permite localizar un problema en contexto. En otros proyectos podrían interesar párrafos, segmentos alineados, títulos, notas al pie o unidades terminológicas. Elegir la unidad de análisis es una decisión lingüística antes que técnica.

Unidad	Ventaja	Limitación
Texto completo	Permite recuentos globales	Puede ocultar dónde ocurre el fenómeno
Párrafo	Mantiene contexto discursivo	Puede ser demasiado amplio para revisión puntual
Oración	Facilita localización y comparación	La segmentación automática puede fallar
Palabra	Permite frecuencias y concordancias	No capta siempre expresiones complejas
Patrón	Localiza estructuras específicas	Depende de una formulación precisa

1.6. Segmentación en oraciones

Segmentar un texto consiste en dividirlo en unidades menores. En la unidad se trabaja con una segmentación muy simple basada en el punto: `texto.split(".")`. Este procedimiento es comprensible para empezar, pero tiene limitaciones evidentes. No todos los puntos cierran oraciones: pueden aparecer en abreviaturas, números, siglas, iniciales o direcciones. Además, una oración puede terminar con signos de interrogación, exclamación o puntos suspensivos.

La segmentación de oraciones es una tarea clásica del procesamiento del lenguaje natural. Kiss y Strunk (2006) mostraron que incluso la detección de límites oracionales puede tratarse con métodos no supervisados y multilingües, precisamente porque no es tan simple como buscar puntos. Los manuales generales de PLN también la presentan como parte de la normalización y el preprocesamiento textual (Bird et al., 2009; Jurafsky & Martin, 2025).

En una unidad inicial de programación para lingüistas, usar `split(".")` tiene valor didáctico: permite ver que una cadena puede convertirse en una lista de segmentos y que cada segmento puede limpiarse con `strip`. Lo importante es acompañar el procedimiento con una advertencia: es una aproximación, no un

segmentador robusto. Cuando el proyecto sea real y de mayor escala, convendrá usar herramientas especializadas o reglas más cuidadosas.

La segmentación también afecta a la interpretación. Si el programa localiza un doble espacio dentro de la oración 4, la revisora puede ir directamente a esa zona. Si solo se informa de que hay tres dobles espacios en todo el texto, el dato es menos útil. Por eso segmentar no es solo dividir: es hacer que los resultados sean revisables.

1.7. Frecuencia léxica y coherencia terminológica

La frecuencia léxica mide cuántas veces aparece una palabra o expresión. En corpus lingüísticos, las frecuencias permiten observar patrones de uso, comparar variedades, estudiar colocaciones o identificar rasgos de un dominio. En revisión lingüística, los recuentos pueden funcionar como indicios de coherencia o divergencia terminológica.

Si una versión usa «equipo» diez veces y otra usa «grupo» diez veces para referirse a la misma entidad, puede haber una diferencia de criterio. Si dentro de una misma versión alternan «compañía», «empresa» y «sociedad» sin justificación, puede haber un problema de coherencia terminológica. Sin embargo, el recuento no decide por sí solo. Puede haber sinónimos legítimos, diferencias de contexto o términos que no sean equivalentes.

La normalización a minúsculas con `lower` facilita comparaciones básicas, porque permite contar «Empresa» y «empresa» como la misma forma. Pero también implica una decisión: se ignora la diferencia entre mayúscula y minúscula. En muchos recuentos esto es útil; en otros puede ser problemático, por ejemplo si se comparan nombres propios, siglas o usos institucionales.

Decisión	Ejemplo	Efecto
Contar con mayúsculas	<code>texto.count("Empresa")</code>	Distingue entre mayúscula y minúscula
Contar en minúsculas	<code>texto.lower().count("empresa")</code>	Agrupar variantes de capitalización
Contar una palabra aislada	«empresa»	Puede incluir parte de formas si no se controla el contexto
Contar expresión	«equipo técnico»	Más preciso para unidades pluriverbales

En proyectos profesionales, la coherencia terminológica suele relacionarse con glosarios, memorias de traducción, bases terminológicas y guías de estilo. Python no sustituye esos recursos, pero puede ayudar a comprobar si una versión respeta una lista de términos o si dos versiones toman decisiones divergentes.

1.8. Patrones formales: espacios, puntuación y normalización

Los patrones formales son señales de superficie: espacios dobles, puntuación pegada, saltos de línea extraños, caracteres invisibles o inconsistencias de formato. A menudo no requieren un análisis lingüístico profundo para detectarse, pero sí afectan a la calidad percibida del texto y a su preparación para otros procesos.

Un ejemplo claro es la ausencia de espacio después de coma o punto: `empresa,el` o `temporales.Utilizando`. En español escrito, lo normal es que después de una coma o un punto haya un espacio si continúa otra palabra, salvo contextos específicos como números, direcciones web, abreviaturas o ciertos formatos técnicos. La detección automática debe tener en cuenta que una regla demasiado simple puede señalar casos legítimos.

Los espacios dobles son otro ejemplo. En muchas versiones finales de documentos, un doble espacio entre palabras es una señal mecánica de falta de limpieza. Sin embargo, en algunos contextos preformateados, tablas o código puede haber espacios intencionales. De nuevo, la regla detecta una señal; la revisión decide.

La normalización textual puede incluir pasar a minúsculas, eliminar espacios sobrantes, sustituir saltos de línea o unificar comillas. Pero normalizar demasiado puede borrar información relevante. En revisión lingüística conviene distinguir entre normalización para analizar y corrección del texto original. Una cosa es crear una copia en minúsculas para contar; otra muy distinta es modificar el documento final.

1.9. Expresiones regulares como reglas transparentes

Las expresiones regulares son patrones formales que permiten buscar secuencias de caracteres con cierta flexibilidad. El módulo `re` de Python implementa operaciones de búsqueda, coincidencia, sustitución y extracción basadas en estos patrones (Python Software Foundation, 2026a). En PLN, las expresiones regulares se usan para tareas como localizar fechas, abreviaturas, formatos, terminaciones, espacios anómalos o construcciones textuales relativamente previsibles (Bird et al., 2009; Friedl, 2006).

Una expresión regular no «comprende» el texto. Lo que hace es comprobar si una secuencia encaja con una forma. Por ejemplo, el patrón `[.,][^\s]` busca un punto o una coma seguido de un carácter que no sea espacio. Eso puede localizar casos como `texto.Sin o empresa,el`. El patrón es transparente: podemos explicar qué significa cada parte. Esa transparencia es valiosa en contextos docentes y profesionales, porque la regla puede discutirse, corregirse y documentarse.

Elemento	Significado aproximado	Ejemplo
<code>[.,]</code>	Un punto o una coma	<code>. o ,</code>
<code>\s</code>	Un espacio o carácter de espacio en blanco	espacio, tabulación, salto de línea
<code>[^\s]</code>	Un carácter que no es espacio	<code>a, E, 5</code>
<code>\w+</code>	Una o más letras, números o guiones bajos	<code>revisar, texto1</code>
<code>\b</code>	Límite de palabra	Final de revisar
<code>`(ar   er   ir)`</code>	Una de varias terminaciones	<code>ar, er, ir</code>

La ventaja de las expresiones regulares es su precisión formal. La limitación es que esa precisión no equivale a interpretación lingüística. Un patrón puede encontrar todas las formas acabadas en `ando` o `iendo`, pero no puede decidir automáticamente si el gerundio está bien usado en cada contexto. Por eso la unidad las presenta como herramientas para generar candidatos a revisión, no como jueces gramaticales.

1.10. Posibles calcos y estructuras con a + infinitivo

En español, ciertas secuencias de sustantivo seguido de a + infinitivo, como «temas a tratar» o «problemas a resolver», han sido tradicionalmente comentadas en obras normativas y de estilo. La valoración depende del contexto, de la estructura exacta y del grado de asentamiento de la expresión. Las obras académicas panhispánicas recomiendan evitar usos innecesarios cuando exista una alternativa más natural, pero no toda aparición de a + infinitivo es incorrecta (Real Academia Española & ASALE, 2005/2023).

La unidad trabaja con un patrón más amplio, por ejemplo formas como «decidió a revisar» o «intentó a implementar», que pueden sonar anómalas en español estándar según el verbo y el régimen exigido. Desde el punto de vista computacional, lo relevante es que muchas de estas secuencias comparten una forma superficial: una palabra, la preposición a y un infinitivo. Ese parecido formal permite buscarlas con una expresión regular.

Ahora bien, el patrón no debe confundirse con una norma. El español tiene muchas construcciones legítimas con a + infinitivo: «empezó a revisar», «volvió a leer», «ayudó a preparar», «salió a comprar». Un detector automático que busque cualquier palabra seguida de a y un infinitivo encontrará tanto casos aceptables como casos dudosos. Por eso el resultado debe llamarse candidato.

Esta distinción es fundamental para estudiantes de lingüística. El programa no sabe régimen verbal, estructura argumental ni aceptabilidad contextual. Solo observa forma. La persona lingüista interpreta si esa forma merece revisión a partir de gramática, uso, estilo y finalidad comunicativa.

1.11. Gerundios como candidatos a revisión

El gerundio en español es una forma no personal del verbo que puede expresar simultaneidad, modo, causa u otras relaciones, según el contexto. La gramática académica describe usos aceptables y usos problemáticos, entre ellos ciertos gerundios de posterioridad o usos adjetivales no asentados (Real Academia Española & ASALE, 2009). En revisión lingüística, los gerundios suelen revisarse porque pueden generar ambigüedad, encadenamiento excesivo o interferencias de estilo.

Detectar gerundios automáticamente parece fácil: muchas formas terminan en -ando, -iendo o -yendo. Sin embargo, una búsqueda por terminación no basta para valorar el uso. «El equipo revisó el informe, incorporando las observaciones» puede requerir lectura contextual; «entró sonriendo» no plantea el mismo tipo de problema; «leyendo el documento, encontró una incoherencia» funciona de otra manera. La forma no determina por completo la función.

La unidad usa expresiones regulares para localizar posibles gerundios y recuperar palabras cercanas. Esto tiene una razón didáctica: el contexto inmediato ayuda a que la revisión sea más útil. No es lo mismo devolver solo utilizando que mostrar «temporales utilizando una herramienta». Cuanto mejor se preserve el contexto, más fácil será decidir si el candidato exige corrección.

El enfoque conecta con una idea general del PLN basado en reglas: muchas tareas empiezan con patrones simples y mejoran cuando incorporan más información. El alumnado debe aprender a valorar ese equilibrio. Una regla muy amplia encuentra muchos casos, pero obliga a revisar más falsos positivos. Una regla muy estrecha encuentra menos ruido, pero puede dejar fuera casos relevantes.

1.12. Falsos positivos, falsos negativos y criterio lingüístico

Un falso positivo aparece cuando el sistema señala un caso que no es realmente problemático. Un falso negativo aparece cuando no señala un caso que sí habría convenido revisar. Estos dos conceptos proceden de la evaluación de sistemas de clasificación, pero son muy útiles para pensar cualquier detector basado en reglas. Si el patrón es muy general, aumentan los falsos positivos. Si el patrón es demasiado restrictivo, aumentan los falsos negativos.

En revisión lingüística, los falsos positivos no son necesariamente un fracaso. Si el volumen es pequeño y el objetivo es no dejar escapar posibles problemas, puede aceptarse una lista más amplia. En cambio, si el volumen es grande y el equipo tiene poco tiempo, conviene diseñar patrones más precisos. La calidad de la herramienta depende de la finalidad del encargo.

Situación	Ejemplo	Consecuencia
Falso positivo	El sistema señala «empezó a revisar» como sospechoso	La persona revisora descarta el caso
Falso negativo	El sistema no detecta una incoherencia terminológica expresada con sinónimo	El problema queda fuera del informe
Regla amplia	Busca cualquier a + infinitivo	Mayor cobertura, más ruido
Regla estrecha	Busca solo una lista de verbos problemáticos	Menos ruido, menor cobertura

La competencia lingüística consiste también en saber diseñar preguntas razonables para la máquina.



Python no debe recibir la orden vaga de «encuentra errores». Debe recibir tareas observables: cuenta este término, busca este patrón, devuelve este contexto, compara estas versiones. Cuanto mejor formulada esté la pregunta, más útil será la respuesta.

1.13. Funciones, informes y trazabilidad

En programación, una función permite encapsular una tarea: recibe datos, aplica una operación y puede devolver un resultado. En una revisión lingüística, las funciones aportan orden y trazabilidad. En lugar de

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

copiar el mismo código varias veces, se crea una comprobación reutilizable: una función para contar términos, otra para detectar espacios dobles, otra para buscar patrones de puntuación, otra para generar un resumen.

La trazabilidad es especialmente importante en entornos profesionales. Si una revisora pregunta cómo se detectaron los casos, el equipo debe poder responder: se usó este patrón, sobre estos textos, con esta normalización. Si el cliente pide repetir la comprobación en una versión nueva, la función permite hacerlo sin rediseñar todo el proceso.

El informe final es la forma comunicativa de esa trazabilidad. No basta con que el código funcione; el resultado debe ser comprensible para una persona que toma decisiones. Un informe útil no solo lista números: explica qué se ha buscado, qué se ha encontrado, dónde aparece y qué cautela interpretativa debe aplicarse.

En Verba Pública, el informe no pretende reemplazar una revisión de estilo completa. Su finalidad es preparar una primera lectura asistida: reducir incertidumbre, ordenar prioridades y documentar indicios. Ese es un uso profesional realista de Python en lingüística aplicada.

1.14. Límites de los métodos basados en reglas

Los métodos basados en reglas son transparentes, fáciles de enseñar y útiles para fenómenos con forma reconocible. Pero tienen límites. El lenguaje natural es variable, ambiguo y dependiente del contexto. Una palabra puede pertenecer a varias categorías; una construcción puede ser aceptable en un contexto y discutible en otro; un sinónimo puede ser deseable o problemático según la política terminológica.

Frente a modelos estadísticos o neuronales, las reglas simples no aprenden de datos ni generalizan semánticamente. Sin embargo, esa limitación también puede ser una ventaja en ciertos encargos. Si se necesita comprobar una convención formal muy concreta, una regla explícita puede ser más adecuada que un sistema opaco. En PLN profesional conviven enfoques distintos: reglas, recursos lingüísticos, corpus, aprendizaje automático y revisión humana.

La unidad se centra deliberadamente en reglas y operaciones básicas porque construyen una base conceptual sólida. Antes de usar bibliotecas avanzadas, conviene entender qué significa leer un texto, dividirlo, recorrerlo, contar, buscar y devolver resultados. Esos fundamentos seguirán siendo necesarios cuando el alumnado trabaje con herramientas más complejas.

2. Análisis automático de textos para revisión lingüística con Python

2.1. Leer textos como datos

Para que Python pueda analizar un documento, primero debe leerlo como texto. En la práctica, eso significa abrir un archivo y convertir su contenido en una cadena de caracteres. La cadena es el tipo de dato básico para trabajar con texto: permite buscar palabras, contar secuencias, dividir por signos, extraer fragmentos y aplicar expresiones regulares.

Un detalle técnico decisivo es la codificación. En español aparecen tildes, eñes, comillas angulares y otros caracteres que deben interpretarse correctamente. Por eso se indica `encoding="utf-8"`. UTF-8 es una codificación estándar ampliamente usada que permite representar caracteres de muchas lenguas. Si se omite la codificación, el sistema puede elegir una por defecto y producir errores o caracteres dañados.

```
with open("texto1.txt", encoding="utf-8") as archivo:
    texto1 = archivo.read()
```

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

El bloque `with` abre el archivo y se encarga de cerrarlo al terminar. La variable `texto1` guarda todo el contenido como una cadena. Desde ese momento, el texto ya puede tratarse como dato lingüístico procesable.

Antes de analizar un documento, Python debe poder leerlo. Un archivo `.txt` contiene texto plano: caracteres sin estilos, sin tablas internas y sin formatos complejos. Por eso resulta muy útil para aprender PLN. Reduce el ruido técnico y permite concentrarse en el contenido lingüístico.

Una forma sencilla de leer un archivo es:

```
texto_a = open("version_a.txt", encoding="utf-8").read()
texto_b = open("version_b.txt", encoding="utf-8").read()
```

Aquí ocurren varias cosas importantes:

- `open(...)` abre el archivo.
- `"version_a.txt"` indica el nombre del archivo.
- `encoding="utf-8"` le dice a Python cómo interpretar caracteres como á, ñ, ü o ¿.
- `.read()` lee todo el contenido y lo convierte en una cadena de texto.
- `texto_a` y `texto_b` son variables que guardan cada versión.

En español, la codificación es especialmente importante. Si se lee mal un archivo, pueden aparecer caracteres extraños: `implementaciÃ³n` en lugar de `implementación`. Ese problema no es lingüístico, pero afecta directamente al análisis. Si Python no reconoce bien las tildes, los conteos pueden fallar.

2.2. Rutas de archivos y `pathlib`

Cuando los documentos están en una carpeta local, conviene trabajar con rutas claras. El módulo `pathlib` permite representar rutas de forma más legible y menos dependiente de detalles del sistema operativo (Python Software Foundation, 2026b). En lugar de escribir una ruta como una cadena larga, se crea un objeto `Path`.

```
from pathlib import Path

carpeta_trabajo = Path("ruta/a/la/carpeta/de/textos")

ruta_a = carpeta_trabajo / "version_a.txt"
ruta_b = carpeta_trabajo / "version_b.txt"

with open(ruta_a, encoding="utf-8") as f:
    texto_a = f.read()

with open(ruta_b, encoding="utf-8") as f:
    texto_b = f.read()
```

Esta forma tiene dos ventajas. La primera es que separa la carpeta de trabajo del nombre de cada archivo. La segunda es que usa `with open(...) as f`, una estructura que abre el archivo, permite leerlo y lo cierra automáticamente al terminar.

Desde el punto de vista del PLN, lo relevante es que cada documento queda representado como un **string**. Un string es una cadena de caracteres. Para Python, un informe institucional, una frase corta o un discurso entero son objetos manipulables siempre que estén guardados como texto.

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

2.3. Cadenas, listas y corpus pequeño

Cuando solo hay dos versiones, es tentador crear variables separadas y repetir el mismo código para cada una:

```
texto_a_lower = texto_a.lower()
texto_b_lower = texto_b.lower()
```

Esto funciona, pero escala mal. Si mañana llegan cincuenta versiones, repetir cincuenta líneas parecidas sería incómodo y propenso a errores.

Una idea fundamental en programación es agrupar elementos relacionados en una **lista**:

```
textos = [texto_a, texto_b]
```

La lista `textos` contiene todas las versiones que queremos analizar. A partir de ese momento, podemos recorrerlas con un bucle:

```
for i, texto in enumerate(textos, start=1):
    print(f"Analizando versión {i}")
```

La función `enumerate()` permite recorrer una lista y, al mismo tiempo, obtener un número de orden. El argumento `start=1` indica que queremos empezar a contar en 1, no en 0. Esto es más natural para informes destinados a personas: **versión 1**, **versión 2**, **versión 3**.

Esta estructura cambia la forma de pensar el análisis. Ya no escribimos una solución para un archivo concreto, sino una solución para cualquier conjunto de textos:

```
textos = [texto_a, texto_b, texto_c]

for i, texto in enumerate(textos, start=1):
    longitud = len(texto)
    print(f"Versión {i}: {longitud} caracteres")
```

En proyectos de revisión, esta generalización es muy valiosa. La misma lógica sirve para dos versiones de una traducción, tres discursos institucionales o cien documentos de una licitación.

2.4. Segmentar en oraciones con `split` y `strip`

La oración es una unidad lingüística central. En revisión automática, segmentar un texto en oraciones permite observar el contenido en fragmentos manejables. No es lo mismo detectar que una palabra aparece en un documento entero que saber en qué oración aparece.

Una estrategia inicial consiste en dividir el texto cada vez que aparece un punto:

```
oraciones_a = texto_a.split(".")
oraciones_b = texto_b.split(".")
```


El método `.split(".")` corta la cadena cada vez que encuentra el carácter ".". El resultado es una lista:

```
for i, oracion in enumerate(oraciones_a[:3], start=1):
    print(f"{i}. {oracion.strip()}")
```

El método `.strip()` elimina espacios al principio y al final del fragmento. Es útil porque, después de cortar por puntos, muchas oraciones empiezan con un espacio.

Esta segmentación es sencilla y comprensible, pero no perfecta. En español, el punto no siempre marca el final de una oración. Puede aparecer en abreviaturas, cifras, iniciales, direcciones web o enumeraciones:

Caso	Problema
Sr. García	El punto no cierra una oración
art. 15	El punto pertenece a una abreviatura
3.2 millones	El punto puede formar parte de una cifra o sección
www.ejemplo.org	El punto aparece dentro de una dirección

Para un primer análisis de control de calidad, `split(".")` puede ser suficiente. Para herramientas profesionales más precisas, convendría usar segmentadores especializados, como los que ofrecen bibliotecas de PLN.

Lo importante es entender la decisión lingüística que hay detrás: segmentar es elegir qué cuenta como unidad de análisis.

2.5. Contar palabras y normalizar con minúsculas

Contar palabras es una de las operaciones más simples y, al mismo tiempo, más útiles del análisis textual. En un contexto de revisión, la frecuencia permite detectar preferencias terminológicas, cambios de estilo o posibles incoherencias.

Supongamos que queremos saber cuántas veces aparece la palabra `implementación` en dos versiones:

```
texto_a_lower = texto_a.lower()
texto_b_lower = texto_b.lower()

conteo_a = texto_a_lower.count("implementación")
conteo_b = texto_b_lower.count("implementación")

print(f"Versión A: {conteo_a} apariciones")
print(f"Versión B: {conteo_b} apariciones")
```

El método `.lower()` convierte todo el texto a minúsculas. Así, `Implementación` al inicio de una oración y `implementación` en medio de una frase se cuentan igual.

Sin esta normalización, Python distinguiría entre:

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

- Implementación
- implementación
- IMPLEMENTACIÓN

Desde el punto de vista lingüístico, pueden ser la misma unidad léxica. Desde el punto de vista computacional, son cadenas distintas. Normalizar el texto permite acercar la representación computacional a la pregunta lingüística.

Ahora bien, `.count("implementación")` tiene límites. Cuenta apariciones exactas de esa secuencia de caracteres. Eso significa que no agrupa automáticamente variantes como:

- implementar
- implementado
- implementaciones
- implementativa

Para agrupar formas relacionadas haría falta lematización o análisis morfológico. En esta unidad trabajamos con conteo exacto porque es transparente y fácil de interpretar.

2.6. Recorrer varias versiones con enumerate

El análisis mejora cuando dejamos de pensar en una pareja fija de textos y empezamos a pensar en un conjunto variable de versiones:

```
textos = [texto_a, texto_b, texto_c]
palabra = "implementación"

for i, texto in enumerate(textos, start=1):
    texto_lower = texto.lower()
    conteo = texto_lower.count(palabra)
    print(f"Versión {i}: {conteo} apariciones de '{palabra}'")
```

Esta estructura contiene tres ideas esenciales:

- `textos` agrupa todos los documentos.
- `for` repite el mismo análisis para cada documento.
- `enumerate()` permite etiquetar cada salida con un número de versión.

El resultado puede servir para detectar diferencias llamativas:

Versión	Apariciones implementación	de	Interpretación inicial
1	8		Alta preferencia por el término
2	1		Posible uso de alternativa terminológica
3	0		Puede haber sustitución por otro concepto o paráfrasis

La interpretación no debe precipitarse. Que una versión use menos un término no significa necesariamente que sea peor. Puede ser más concisa, haber reformulado el contenido o emplear un término recomendado. El conteo solo abre una pregunta.

2.7. Detectar ausencia de espacio después de puntuación

Uno de los problemas formales más fáciles de detectar automáticamente es la ausencia de espacio después de punto o coma:

Fragmento	Problema
El informe fue aprobado.La comisión lo publicó.	Falta espacio tras punto
El proyecto incluye formación,seguimiento y evaluación.	Falta espacio tras coma

Estos errores parecen pequeños, pero en documentos institucionales afectan a la calidad percibida. También pueden dificultar procesos automáticos posteriores, como la segmentación o la tokenización.

Para detectarlos, se puede usar una expresión regular:

```
import re

errores = [m.start() for m in re.finditer(r"[.,][^\s]", texto_a)]
print(f"Número de signos sin espacio después: {len(errores)}")
```

La expresión `r"[.,][^\s]"` se lee así:

Parte	Significado
<code>r"..."</code>	Cadena cruda, útil para escribir patrones de expresiones regulares
<code>[.,]</code>	Un punto o una coma
<code>[^\s]</code>	Un carácter que no sea espacio

Por tanto, el patrón busca un punto o una coma seguidos inmediatamente de un carácter que no sea espacio.

El método `re.finditer()` devuelve coincidencias con información de posición. Por eso podemos usar `m.start()` para saber en qué índice del texto empieza cada caso.

2.8. Recuperar contexto con índices y slicing

Un informe que solo dice «hay 12 errores» no es suficiente. La persona que revisa necesita saber dónde están. Para eso se extraen fragmentos de contexto:

```
for pos in errores:
```

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

```
inicio = max(pos - 20, 0)
fin = pos + 20
fragmento = texto_a[inicio:fin]
print(f"...{fragmento}...")
```

Este fragmento introduce varias ideas:

- pos es la posición donde se detectó el patrón.
- pos - 20 toma veinte caracteres anteriores.
- pos + 20 toma veinte caracteres posteriores.
- max(pos - 20, 0) evita que el índice inicial sea negativo.
- texto_a[inicio:fin] extrae una porción del texto.

El resultado permite ver algo como:

```
...evaluación.La comisión aprobó...
...formación,seguimiento y apoyo...
```

La extracción de contexto es una práctica habitual en revisión automática. No basta con encontrar el error; hay que mostrarlo de forma útil.

2.9. Localizar dobles espacios

Los dobles espacios son otro problema formal frecuente. Suelen aparecer al editar, copiar y pegar o combinar fragmentos procedentes de distintas fuentes:

```
La institución  publicará el informe final.
```

Una detección mínima puede hacerse con el operador in:

```
hay_dobles_espacios = "  " in texto_a
print(hay_dobles_espacios)
```

La expresión " " in texto_a pregunta si la secuencia de dos espacios aparece en el texto. Devuelve True o False.

Para contar cuántas veces aparece:

```
total = texto_a.count("  ")
print(f"Dobles espacios detectados: {total}")
```

Y para saber en qué oraciones se encuentran:

```
dobles = [
    i + 1
    for i, oracion in enumerate(oraciones_a)
    if "  " in oracion
]
```

Los derechos de este material pertenecen a la Dr.ª Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

```
print(f"Aparecen en las oraciones: {dobles}")
```

Esta estructura es una **lista por comprensión**. Construye una lista nueva a partir de una condición. En este caso, guarda los números de las oraciones que contienen dobles espacios.

La parte $i + 1$ se usa porque Python empieza a contar en 0, pero en un informe para personas suele ser más natural empezar por 1.

2.10. Comparación terminológica básica

Cuando se trabaja con varios textos, también conviene guardar las oraciones de cada texto en una lista:

```
textos = [texto_a, texto_b, texto_c]
oraciones_textos = [oraciones_a, oraciones_b, oraciones_c]
```

Para recorrer al mismo tiempo cada texto y su lista de oraciones, se puede usar `zip()`:

```
for i, (texto, oraciones) in enumerate(zip(textos, oraciones_textos), start=1):
    dobles = [
        n + 1
        for n, oracion in enumerate(oraciones)
        if " " in oracion
    ]
    print(f"Versión {i}: {texto.count(' ')} dobles espacios")
    print(f"Oraciones afectadas: {dobles}")
```

`zip(textos, oraciones_textos)` empareja los elementos de ambas listas:

Elemento de textos	Elemento de oraciones_textos
texto_a	oraciones_a
texto_b	oraciones_b
texto_c	oraciones_c

Esta técnica es muy útil cuando varias estructuras de datos representan distintos niveles del mismo corpus: texto completo, oraciones, metadatos, proveedor, fecha de entrega o estado de revisión.

La comparación terminológica es el corazón profesional de esta unidad. En textos paralelos monolingües, muchas divergencias no son errores gramaticales, sino diferencias de elección léxica.

Supongamos que queremos comparar estas parejas:

```
parejas_terminos = [
    ("equipo", "grupo"),
    ("compañía", "empresa"),
    ("gerente", "director"),
    ("procesos", "procedimientos"),
```

]

Cada pareja representa dos formas que pueden competir dentro de una misma política terminológica. Una institución puede considerar que ambas son aceptables en contextos distintos, o puede preferir una.

Un análisis sencillo cuenta cada término en cada versión:

```
texto_a_lower = texto_a.lower()
texto_b_lower = texto_b.lower()

for termino_1, termino_2 in parejas_terminos:
    c1_a = texto_a_lower.count(termino_1)
    c2_a = texto_a_lower.count(termino_2)
    c1_b = texto_b_lower.count(termino_1)
    c2_b = texto_b_lower.count(termino_2)

    print(f"{termino_1}: versión A={c1_a}, versión B={c1_b}")
    print(f"{termino_2}: versión A={c2_a}, versión B={c2_b}")
```

El resultado puede revelar situaciones como estas:

Situación	Posible interpretación
Una versión usa equipo y otra grupo	Divergencia terminológica entre proveedores
Una versión alterna empresa y compañía	Falta de uniformidad interna
Un término recomendado no aparece	Posible incumplimiento de glosario
Un término aparece en exceso	Puede haber traducción literal, repetición o preferencia estilística

La tabla no decide por sí sola qué término es mejor. Proporciona evidencia para discutirlo.

2.11. Tablas de resultados

En revisión profesional, la forma de presentar los resultados importa. Una salida desordenada puede ser difícil de interpretar. Python permite alinear texto y números mediante cadenas formateadas.

```
print(f"{'Término':<20} | {'Versión 1':^10} | {'Versión 2':^10}")
print("-" * 48)

for termino_1, termino_2 in parejas_terminos:
    print(f"{termino_1:<20} | {c1_a:^10} | {c1_b:^10}")
    print(f"{termino_2:<20} | {c2_a:^10} | {c2_b:^10}")
    print("-" * 48)
```

Las expresiones entre llaves controlan el formato:

Código	Significado
{termino_1:<20}	Escribe el término alineado a la izquierda en 20 caracteres
{c1_a:^10}	Centra el número en 10 caracteres
"-" * 48	Repite el guion 48 veces para crear una línea separadora

Este tipo de formato no es solo estético. Ayuda a leer la comparación como una tabla. Cuando los resultados se entregan a un equipo editorial, claridad visual y precisión técnica van juntas.

2.12. Comparar términos en muchas versiones

La comparación puede generalizarse a cualquier número de textos:

```
textos = [texto_a, texto_b, texto_c]
textos_lower = [texto.lower() for texto in textos]

cabecera = "Término".ljust(20)
for i in range(len(textos_lower)):
    cabecera += f" | Versión {i + 1:^8}"
print(cabecera)

for pareja in parejas_terminos:
    for termino in pareja:
        fila = f"{termino:<20}"
        for texto in textos_lower:
            fila += f" | {texto.count(termino):^10}"
        print(fila)
```

Aquí aparece una nueva lista por comprensión:

```
textos_lower = [texto.lower() for texto in textos]
```


Esta línea crea una lista con todas las versiones en minúsculas. Es una forma compacta de preparar los datos antes de contarlos.

La comparación en muchas versiones permite detectar patrones más ricos:

Término	Versión 1	Versión 2	Versión 3
adquirir	4	0	1
obtener	0	5	2
conseguir	1	0	6
trabajador	8	1	0
empleado	0	6	2
asalariado	0	0	5

Una tabla así invita a formular hipótesis:

- Quizá cada proveedor tiene una preferencia léxica estable.
- Quizá una versión es más técnica y otra más general.
- Quizá se están usando términos que no pertenecen al mismo registro.
- Quizá hay que consolidar un glosario antes de publicar.

El análisis cuantitativo no cierra la interpretación; la abre de forma ordenada.

2.13. Posibles galicismos: Buscar candidatos a + infinitivo

En español normativo, una construcción como **temas a tratar** suele citarse como posible calco del francés cuando funciona como sustantivo + a + infinitivo. En muchos contextos se recomienda reformular con estructuras como:

Construcción revisable	Posible alternativa
asuntos a debatir	asuntos que se deben debatir
tareas a realizar	tareas que hay que realizar
documentos a presentar	documentos que deben presentarse

No obstante, el diagnóstico no puede automatizarse con una regla tan simple como «toda secuencia a + infinitivo es incorrecta». En español hay muchísimas construcciones correctas con a + infinitivo:

- vamos a revisar
- empezó a trabajar
- aprendió a programar
- ayuda a entender

Por eso conviene hablar de **posibles galicismos** o **candidatos a revisión**, no de errores automáticos.

Un patrón inicial puede localizar secuencias de palabra + a + infinitivo:

```
import re

patron = r"(\w+)\s+a\s+(\w+(ar|er|ir))\b"
resultados = re.findall(patron, texto_a.lower())

pares = [(anterior, verbo) for anterior, verbo, _ in resultados]

for anterior, verbo in pares:
    print(f"{anterior} a {verbo}")
```

La lista pares conserva solo la palabra anterior y el verbo. El tercer elemento de cada coincidencia, la terminación ar, er o ir, se descarta con `_` porque no nos interesa para el informe.

Este análisis es útil como filtro inicial. Si aparecen muchos casos, la lingüista puede revisar el contexto y distinguir:

Caso detectado	Lectura posible
documentos a presentar	Candidato a revisión
vamos a presentar	Construcción verbal correcta
ayuda a presentar	Régimen verbal correcto
aspectos a considerar	Candidato a revisión según estilo

Para automatizar mejor esta tarea haría falta combinar varias técnicas: tokenización, lematización y etiquetado morfosintáctico. En particular, habría que comprobar si la palabra anterior es un sustantivo y si la secuencia forma realmente una construcción nominal.

2.14. Buscar gerundios y contexto inmediato

El gerundio en español tiene usos correctos y usos problemáticos. Puede expresar simultaneidad, modo o proceso:

- El equipo revisó el informe siguiendo las instrucciones.
- La comisión analizó los datos comparando las dos versiones.

Pero también puede aparecer en usos desaconsejados, como el gerundio de posterioridad o ciertos gerundios especificativos:

Tipo	Fragmento revisable	Motivo
Posterioridad	Se aprobó el informe, publicándose al día siguiente.	El gerundio expresa una acción posterior
Especificativo	Se envió una carta informando del cambio.	Puede funcionar como modificador poco natural

Una regla formal inicial consiste en buscar palabras terminadas en -ando, -iendo o -yendo:

```
import re

patron = r"(\w+)?\s*(\w+(ando|iendo|yendo))\s*(\w+)?"
coincidencias = re.findall(patron, texto_a.lower())

gerundios = [
    (anterior if anterior else "", gerundio, siguiente if siguiente else "")
    for anterior, gerundio, _, siguiente in coincidencias
]

for anterior, gerundio, siguiente in gerundios:
    print(anterior, gerundio, siguiente)
```

Este patrón busca el gerundio y captura, si existen, una palabra anterior y una palabra siguiente. El contexto mínimo ayuda a decidir si el caso requiere revisión.

De nuevo, la herramienta no declara automáticamente que un gerundio sea incorrecto. Solo localiza formas que una persona puede evaluar. La diferencia entre detectar y diagnosticar es central:

Detección automática	Diagnóstico lingüístico
Encuentra palabras terminadas en -ando, -iendo, -yendo	Decide si el uso del gerundio es aceptable
Muestra contexto inmediato	Interpreta relación temporal, sintáctica y semántica
Cuenta ocurrencias	Valora si hay patrón estilístico problemático

Como se explicó en el marco teórico, un **falso positivo** es un caso que la herramienta marca como sospechoso, pero que no constituye realmente un problema.

En análisis de revisión lingüística, los falsos positivos son inevitables cuando usamos reglas simples. Por ejemplo:

```
patron = r"(\w+)\s+a\s+(\w+(ar|er|ir))\b"
```

Este patrón puede detectar:

- vamos a revisar
- ayuda a entender
- documentos a presentar

Solo algunos casos serán revisables como posible calco. El patrón no sabe si la palabra anterior es un sustantivo, un verbo conjugado o una forma auxiliar. Tampoco conoce el régimen verbal.

Esto no invalida el análisis. Al contrario: enseña una lección metodológica importante. En PLN, muchas herramientas funcionan por aproximación. Una buena herramienta no es necesariamente la que no se equivoca nunca, sino la que reduce el trabajo humano sin ocultar sus límites.

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

Para una lingüista, el objetivo no es aceptar ciegamente la salida del programa. El objetivo es aprender a preguntar:

- ¿Qué está detectando exactamente esta regla?
- ¿Qué casos deja fuera?
- ¿Qué casos marca indebidamente?
- ¿Qué conocimiento lingüístico haría falta añadir?
- ¿Qué coste tendría mejorar la precisión?

Esta actitud crítica diferencia el uso superficial de Python de un verdadero análisis lingüístico computacional.

2.15. Organizar el análisis con funciones

Cuando un análisis crece, conviene organizarlo en **funciones**. Una función es un bloque de código con nombre que realiza una tarea concreta.

Por ejemplo:

```
def contar_palabra(texto, palabra):
    texto_lower = texto.lower()
    return texto_lower.count(palabra)
```

Esta función recibe un texto y una palabra. Convierte el texto a minúsculas y devuelve el número de apariciones.

Podemos usarla así:

```
conteo = contar_palabra(texto_a, "implementación")
print(conteo)
```

Las funciones aportan varias ventajas:

Ventaja	Explicación
Organización	Cada tarea tiene un nombre claro
Reutilización	La misma función sirve para otros textos
Legibilidad	El informe final puede llamar funciones en orden
Mantenimiento	Si se mejora una regla, se cambia en un solo lugar
Escalabilidad	Es más fácil añadir nuevas comprobaciones

Sin funciones, el código puede convertirse en una sucesión larga de instrucciones repetidas. Con funciones, el análisis se parece más a un conjunto ordenado de herramientas.

Un sistema básico de revisión puede organizarse en funciones como estas:

```
def contar_palabra(texto, palabra):
    return texto.lower().count(palabra)
```

```
def buscar_falta_espacio(texto):
    import re
    return [m.start() for m in re.finditer(r"[.,][^\s]", texto)]
```

```
def detectar_dobles_espacios(texto, oraciones):
    posiciones = [
        i + 1
        for i, oracion in enumerate(oraciones)
        if " " in oracion
    ]
    return texto.count(" "), posiciones
```

```
def analizar_a_infinitivo(texto):
    import re
    patron = r"(\w+)\s+a\s+(\w+(ar|er|ir))\b"
    resultados = re.findall(patron, texto.lower())
    return [(anterior, verbo) for anterior, verbo, _ in resultados]
```

```
def buscar_gerundios(texto):
    import re
    patron = r"(\w+)?\s*(\w+(ando|iendo|yendo))\s*(\w+)?"
    coincidencias = re.findall(patron, texto.lower())
    return [
        (anterior if anterior else "", gerundio, siguiente if siguiente else "")
        for anterior, gerundio, _, siguiente in coincidencias
    ]
```

Cada función resuelve una pregunta concreta:

Función	Pregunta que responde
contar_palabra()	¿Cuántas veces aparece este término?
buscar_falta_espacio()	¿Dónde falta espacio tras punto o coma?
detectar_dobles_espacios()	¿Cuántos doubles espacios hay y en qué oraciones?
analizar_a_infinitivo()	¿Qué secuencias pueden requerir revisión como a + infinitivo?
buscar_gerundios()	¿Qué gerundios aparecen y con qué contexto mínimo?

Los derechos de este material pertenecen a la Dr.^a Blanca Hernández Pardo. Queda prohibida la difusión, distribución o reproducción total o parcial de este material sin autorización expresa de la autora.

El nombre de una función debe ser transparente. Una persona que lea `buscar_gerundios(texto)` entiende la intención sin tener que estudiar cada línea interna.

2.16. Devolver resultados con `return`

Una función puede imprimir algo en pantalla o puede devolver un resultado. No es lo mismo.

Observa esta función:

```
def sumar(a, b):  
    resultado = a + b
```

Calcula `a + b`, pero no devuelve el resultado. Si se intenta guardar la salida:

```
x = sumar(3, 4)  
print(x)
```

Python imprimirá:

```
None
```

None significa que la función no ha devuelto ningún valor.

Para que el resultado pueda usarse fuera de la función, necesitamos `return`:

```
def sumar(a, b):  
    resultado = a + b  
    return resultado
```

O, de forma más breve:

```
def sumar(a, b):  
    return a + b
```

Entonces:

```
x = sumar(3, 4)  
print(x)
```

produce:

```
7
```

En un informe de revisión lingüística, return es importante porque permite combinar resultados. Una función puede contar términos y devolver números; otra puede detectar errores y devolver posiciones; otra puede construir una tabla. Si solo imprimen, los resultados se ven en pantalla, pero es más difícil reutilizarlos.

2.17. Generar un informe final

El objetivo práctico no es acumular fragmentos de código, sino producir un informe ordenado. Un informe final puede organizarse por secciones:

- Frecuencia de términos clave.
- Ausencias de espacio tras puntuación.
- Dobles espacios.
- Comparación terminológica.
- Posibles estructuras a + infinitivo.
- Gerundios localizados.
- Observaciones para revisión humana.

Un esquema simplificado podría ser:

```
textos = [texto_a, texto_b]
oraciones_textos = [oraciones_a, oraciones_b]

print("INFORME DE REVISIÓN LINGÜÍSTICA")
print("=" * 40)

for i, texto in enumerate(textos, start=1):
    print(f"\nVERSIÓN {i}")
    print("-" * 20)

    conteo = contar_palabra(texto, "implementación")
    print(f"Frecuencia de 'implementación': {conteo}")

    errores = buscar_falta_espacio(texto)
    print(f"Signos sin espacio después: {len(errores)}")

    total_dobles, oraciones = detectar_dobles_espacios(
        texto,
        oraciones_textos[i - 1]
    )
    print(f"Dobles espacios: {total_dobles}")
    print(f"Oraciones afectadas: {oraciones}")

    candidatos = analizar_a_infinitivo(texto)
    print(f"Posibles casos de 'a + infinitivo': {len(candidatos)}")

    gerundios = buscar_gerundios(texto)
    print(f"Gerundios encontrados: {len(gerundios)}")
```

Este informe no pretende ser el documento final que se entregaría a una institución. Es una base. A partir de él, se podrían añadir:

- Exportación a .csv o .xlsx.
- Resúmenes por proveedor.
- Fragmentos de contexto para cada hallazgo.
- Clasificación manual de casos correctos e incorrectos.
- Gráficos de distribución terminológica.
- Integración con glosarios institucionales.

La idea clave es que Python organiza la evidencia. La decisión profesional sigue perteneciendo a la especialista.

2.18. Cómo leer los resultados antes de corregir

Una buena práctica consiste en leer los resultados con tres preguntas:

- ¿Qué regla produjo este hallazgo?
- ¿Qué parte del texto se muestra como contexto?
- ¿Qué decisión humana hace falta ahora?

Estas preguntas evitan una confianza excesiva en la salida automática. También ayudan a documentar el trabajo. Si una incidencia se descarta, puede anotarse por qué. Si se confirma, puede corregirse con una justificación. En entornos profesionales, esta trazabilidad puede ser tan importante como la corrección misma.

Reglas simples frente a análisis lingüístico avanzado

Las técnicas de esta unidad se basan en reglas simples:

- Cortar por puntos.
- Contar cadenas exactas.
- Buscar patrones con expresiones regulares.
- Identificar terminaciones como -ando, -iendo o -yendo.

Estas reglas son transparentes. Podemos explicar exactamente qué hacen. Esa transparencia es una virtud en formación y en revisión profesional. Sin embargo, también tienen límites:

Regla simple	Límite
<code>split(".")</code>	Confunde puntos de abreviaturas con finales de oración
<code>.count("empresa")</code>	Puede contar partes de palabras o no agrupar variantes flexivas
Patrón a + infinitivo	No distingue sustantivos de verbos conjugados
Terminaciones de gerundio	No interpreta la función sintáctica del gerundio
Conteo de sinónimos	No sabe si dos términos son equivalentes en contexto

Para mejorar la precisión, se pueden incorporar técnicas más avanzadas:

- **Tokenización**, para separar el texto en unidades lingüísticas más precisas.
- **Lematización**, para agrupar formas como trabaja, trabajó, trabajar.
- **Etiquetado morfosintáctico**, para saber si una palabra es sustantivo, verbo, adjetivo, etc.
- **Análisis de dependencias**, para estudiar relaciones sintácticas.
- **Reconocimiento de entidades**, para detectar instituciones, lugares o personas.
- **Modelos estadísticos o neuronales**, para capturar patrones no formulados manualmente.

La progresión natural en PLN consiste en empezar por reglas simples, comprender sus límites y avanzar hacia herramientas más sofisticadas cuando la pregunta lo requiere.

Buenas prácticas para revisar textos con Python

Un análisis automático de revisión lingüística debe ser claro, reproducible y prudente. Algunas buenas prácticas son:

- Nombrar bien las variables. `texto_a` es más claro que `x`.
- Guardar juntos los textos relacionados. Una lista facilita el análisis conjunto.
- Normalizar cuando sea necesario. `.lower()` evita diferencias irrelevantes entre mayúsculas y minúsculas.
- Conservar contexto. Un hallazgo sin fragmento puede ser difícil de corregir.
- Distinguir detección de diagnóstico. Que una regla encuentre algo no significa que sea un error.
- Presentar resultados con etiquetas claras. El informe debe poder leerlo otra persona.
- Evitar copiar y pegar bloques repetidos. Las funciones reducen errores y mejoran la reutilización.
- Documentar los límites. Toda herramienta debe decir qué detecta y qué no.
- Mantener el criterio lingüístico en el centro. Python apoya la revisión; no sustituye la interpretación.

Estas prácticas son tan importantes como la sintaxis. En entornos profesionales, un análisis útil no es solo el que produce números, sino el que permite tomar decisiones con confianza.

Errores frecuentes de aprendizaje

- Confundir una coincidencia automática con un error lingüístico confirmado.
- Pensar que `split(".")` segmenta siempre oraciones correctamente.
- Olvidar `encoding="utf-8"` al leer textos con tildes, eñes o comillas españolas.
- Contar términos sin normalizar mayúsculas y minúsculas cuando la comparación lo requiere.
- Usar `count` sin recordar que busca secuencias, no necesariamente palabras completas.
- Interpretar una divergencia terminológica como error sin consultar el glosario o el contexto.
- Creer que una expresión regular comprende gramática, régimen verbal o estilo.
- Diseñar patrones demasiado amplios y no prever falsos positivos.
- Diseñar patrones demasiado estrechos y dejar fuera casos relevantes.
- Usar `print` dentro de todo el código y luego no poder reutilizar los resultados.
- Repetir bloques casi idénticos en lugar de crear funciones.
- Presentar un informe sin contexto suficiente para que una revisora pueda decidir.

Síntesis final

Esta unidad muestra cómo Python puede apoyar la revisión lingüística de textos paralelos monolingües. El objetivo no es automatizar el juicio experto, sino construir una cadena de análisis que convierta los textos en datos observables: se leen archivos, se guardan versiones, se segmentan oraciones, se cuentan términos, se buscan patrones formales y se generan resultados interpretables.

El marco teórico de la unidad conecta tres ámbitos: la revisión y evaluación de calidad en servicios lingüísticos, la lingüística de corpus y el PLN basado en reglas. De la revisión profesional se toma la importancia del criterio y de la trazabilidad. De la lingüística de corpus se toma la idea de observar patrones en colecciones de textos. Del PLN se toman herramientas como cadenas, listas, bucles, expresiones regulares y funciones.

El aprendizaje más importante es metodológico: el código debe formular preguntas concretas. Python puede ayudar a responder «cuántas veces aparece este término», «dónde hay una coma sin espacio posterior» o «qué segmentos contienen una forma terminada en gerundio». No puede responder por sí solo «qué versión es mejor» sin un marco de decisión humano.

Para una alumna o un alumno lingüista, esta unidad ofrece una base profesional muy valiosa. Permite pasar de la lectura lineal a la lectura asistida por evidencias, y de la intuición individual a un procedimiento más documentable. Esa es una competencia central en PLN aplicado a revisión, traducción, edición y control de calidad textual.

Cierre narrativo

Al final de la simulación, Verba Pública no entrega a la institución una sentencia automática. Entrega un informe preliminar de revisión: términos que conviene unificar, patrones formales que deben comprobarse, estructuras candidatas a lectura lingüística y una explicación clara de las cautelas. La revisora sénior conserva la última palabra, pero ya no parte de una página en blanco.

La institución gana tiempo y transparencia. El equipo lingüístico gana una forma de justificar mejor sus recomendaciones. Y el alumnado aprende una lección clave: programar para PLN no consiste solo en escribir código que funcione, sino en diseñar procedimientos que respeten la complejidad del lenguaje y ayuden a tomar mejores decisiones.

Referencias

- Bird, S., Klein, E. y Loper, E. (2009). *Natural language processing with Python*. O'Reilly Media. <https://www.nltk.org/book/>
- Bowker, L. (2002). *Computer-aided translation technology: A practical introduction*. University of Ottawa Press.
- Friedl, J. E. F. (2006). *Mastering regular expressions* (3rd ed.). O'Reilly Media.
- International Organization for Standardization. (2015). *ISO 17100:2015. Translation services: Requirements for translation services*. <https://www.iso.org/standard/59149.html>
- International Organization for Standardization. (2024). *ISO 5060:2024. Translation services: Evaluation of translation output*. <https://www.iso.org/standard/80786.html>
- Jurafsky, D. y Martin, J. H. (2025). *Speech and language processing* (3rd ed. draft). Stanford University. <https://web.stanford.edu/~jurafsky/slp3/>
- Kiss, T. y Strunk, J. (2006). Unsupervised multilingual sentence boundary detection. *Computational Linguistics*, 32(4), 485-525. <https://doi.org/10.1162/coli.2006.32.4.485>

- Lommel, A., Burchardt, A. y Uszkoreit, H. (2014). Multidimensional Quality Metrics: A flexible system for assessing translation quality. *Tradumática*, 12, 455-463. <https://doi.org/10.5565/rev/tradumatica.77>
- McEnery, T. y Hardie, A. (2012). *Corpus linguistics: Method, theory and practice*. Cambridge University Press.
- Olohan, M. (2004). *Introducing corpora in translation studies*. Routledge.
- Python Software Foundation. (2026a). *re: Regular expression operations*. Python 3.14 documentation. <https://docs.python.org/3/library/re.html>
- Python Software Foundation. (2026b). *pathlib: Object-oriented filesystem paths*. Python 3.14 documentation. <https://docs.python.org/3/library/pathlib.html>
- Python Software Foundation. (2026c). *Built-in functions: open*. Python 3.14 documentation. <https://docs.python.org/3/library/functions.html#open>
- Real Academia Española & Asociación de Academias de la Lengua Española. (2005/2023). *Diccionario panhispánico de dudas*. <https://www.rae.es/dpd/>
- Real Academia Española & Asociación de Academias de la Lengua Española. (2009). *Nueva gramática de la lengua española*. Espasa.
- Zanettin, F. (2012). *Translation-driven corpora: Corpus resources for descriptive and applied translation studies*. Routledge.